

Semantic Contracts for HE $\leftrightarrow$ PeTTa Translation

Shared Core, Compatibility Profiles, and Production Boundaries

Zar Goertzel

May 2026

Abstract

This document states the semantic contract for translating between Hyperon Experimental (HE) MeTTa and PeTTa MeTTa. The central point is that both systems use MeTTa syntax, but they do not always give the same operational meaning to every surface form. A faithful translator must therefore do more than parse and reprint S-expressions: it must preserve observable behavior on a shared core, lower source conveniences to explicit target forms where possible, and classify helper or profile-specific surfaces explicitly when portability requires more than syntax.

We describe the shared and non-shared language surfaces, the main translation rules in both directions, the status of quote/eval/reduce, **test**, **hyperpose**, type queries, state, imports, and space operations, and the validation obligations for production use. The companion engineering specification lives with the translator implementation at `hyperon/translators/specs/he-petta-translation-semantics.md`.

Contents

1 Problem	2
2 Semantic Lanes	2
3 HE Surface	3
4 PeTTa Surface	3
5 HE to PeTTa	4
6 PeTTa to HE	4
7 Quote, Eval, Reduce	5
8 Tests And Assertions	5
9 Hyperpose	6
10 Type Queries	6
11 Imports, Spaces, And State	6
12 Evidence And Gates	6

13 Production Criteria	7
14 Conclusion	7

1 Problem

HE and PeTTa are both MeTTa runtimes, but MeTTa syntax alone does not fix all runtime behavior. A file that ends in `.metta` may be:

1. a source program intended for default PeTTa;
2. an HE program intended for a reference HE runtime;
3. a generated HE-facing artifact intended for `petta -he`;
4. an artifact that supplies helper definitions to preserve source behavior;
5. an extension program that relies on host facilities outside core HE.

The translator’s job is to keep these categories explicit. Classifying every generated artifact as pure HE would be incorrect. Classifying every difference as an untranslatable failure would also be incorrect, because many differences have principled lowerings.

Positive example. The PeTTa source form

```
!(test (length (collapse (demo-peano 300))) 301)
```

can be translated to an explicit HE-facing computation:

```
!(test (let $_tr_tuple_1 (collapse (demo-peano 300))
      (size-atom $_tr_tuple_1))
      301)
```

This is ordinary portable lowering.

Negative example. The PeTTa source form

```
!(test (quote (+ 1 2)) (+ 1 2))
```

depends on PeTTa’s visible quote behavior. A target runtime whose `quote` returns a visible wrapper cannot be claimed to agree merely because both sides parse the same text.

2 Semantic Lanes

The translator uses the following semantic lanes.

Lane	Meaning	Example
Shared core	Behavior is expected to agree on the target engines under consideration.	<code>if</code> , <code>case</code> , many <code>let</code> , <code>match</code> , and arithmetic examples.

Lane	Meaning	Example
Portable lowering	Source surface is rewritten to target forms without PeTTa-only assumptions.	length (collapse e) to let plus size-atom.
Translator helper	Generated artifact needs a small helper definition to preserve source behavior.	test, assertEqualToEval, quoted-syntax.
PeTTa HE compatibility	Intended for <code>./run.sh -he</code> ; may rely on profile behavior outside HE core.	Callable-return application and selected quote/eval/reduce compatibility.
PeTTa extension	Useful runtime surface outside the strict portability claim.	Python FFI, host resources, selected path/indexing support.

This table is part of the correctness contract. A helper-class artifact may be correct for its intended target while remaining unsuitable as a backend-agnostic HE conformance example.

3 HE Surface

The HE target surface used by the translator is centered on explicit evaluation and binding constructs. Important target forms include:

Construct	Role
chain	Sequenced evaluation with result binding.
collapse, collapse-bind	Collection of nondeterministic results.
superpose, superpose-bind	Re-entry of alternatives.
eval, evalc, unquote	Explicit evaluation of syntax/data.
case, switch	Pattern-directed branching.
size-atom	Explicit tuple size/count surface.
function, return	Minimal function-return surface.
metta	Interpreter call with expected type and space.

The translator should not turn implementation extensions into HE core requirements. If a target runtime provides an extra surface, that surface must be named as a profile or extension.

4 PeTTa Surface

PeTTa source programs use the same S-expression syntax plus Prolog-backed runtime conveniences. Some forms are shared. Others require explicit translation.

PeTTa surface	Translation status
<code>let</code> , <code>let*</code> , <code>if</code> , <code>case</code> , <code>match</code> , <code>collapse</code> , <code>superpose</code>	Usually shared or nearly identity.
<code>progn</code> , <code>progn1</code>	Lowered to sequenced <code>chain/let</code> forms.
<code>foldall</code> , PeTTa-style <code>foldl-atom</code>	Lowered to <code>collapse</code> plus explicit accumulator/item variables.
<code>length (collapse e)</code>	Lowered to <code>let tuple (collapse e') (size-atom tuple)</code> .
<code>test</code>	Preserved because its printed report is observable behavior.
<code>quote</code> , <code>eval</code> , <code>reduce</code>	Compatibility lane; see Section 7.
<code>hyperpose</code>	Lowered to <code>superpose</code> by default, preserved only with explicit target intent.
Python, Git, and host FFI	Passed through or extension-classified; not pure HE.

5 HE to PeTTa

The HE-to-PeTTa direction adapts explicit HE sequencing and binding to PeTTa source forms:

HE input	PeTTa output	Intent
<code>(chain e \$x b)</code>	<code>(let \$x e b)</code>	Preserve sequencing.
<code>(collapse-bind e)</code>	<code>(collapse e')</code>	Preserve collection.
<code>(superpose-bind xs)</code>	<code>(superpose xs')</code>	Preserve alternatives.
<code>(nop e)</code>	<code>(let \$fresh e' ())</code>	Keep effect, discard result.
<code>(switch x bs)</code>	<code>(case x' bs')</code>	Translate branches.
<code>(function (return x))</code>	<code>x'</code>	Remove wrapper where safe.

Fresh names use a reserved generated namespace such as `$__tr_...`. Correctness requires capture avoidance: generated names must not accidentally bind or shadow source variables.

6 PeTTa to HE

The PeTTa-to-HE direction lowers source conveniences to explicit HE-facing forms:

PeTTa input	HE-facing output	Intent
<code>progn</code>	Nested sequencing	Preserve effects and final value.
<code>prog1</code>	Result capture plus sequencing	Preserve first result.
<code>foldall</code>	<code>collapse</code> plus <code>foldl-atom</code>	Make fold explicit.
<code>foldl-atom</code>	Explicit accumulator/item variables	Avoid implicit agg shape.
<code>length (collapse e)</code>	<code>let</code> plus <code>size-atom</code>	Portable lowering.
<code>test</code>	<code>test</code>	Preserve observable test surface.
<code>unique-atom (collapse e)</code>	<code>collapse (unique e')</code>	Use target uniqueness.
<code>hyperpose</code>	<code>superpose</code> , unless preserved	Drop scheduling, keep values.

7 Quote, Eval, Reduce

Quote/eval/reduce are a common compatibility boundary. In default PeTTa,

```
!(quote (+ 1 2))
```

exposes the syntax value:

```
(+ 1 2)
```

whereas some HE implementations expose a visible quote wrapper:

```
(quote (+ 1 2))
```

The PeTTa-to-HE translator preserves PeTTa-visible behavior by inserting a helper when needed:

```
(= (quoted-syntax (quote $expr)) $expr)
```

and lowering:

```
(quote e) -> (quoted-syntax (quote e'))
(eval e) -> (unquote (quote e'))
(reduce e) -> (unquote (quote e'))
```

This is a compatibility translation, not a pure-HE core claim. `quoted-syntax` is a helper surface. A backend-agnostic artifact must avoid it, supply it explicitly, or replace it with a target-specific quote/eval representation whose semantics are specified.

8 Tests And Assertions

PeTTa's `test` form prints an actual/expected report before returning the assertion result. That output is observable behavior. Therefore the translator preserves source `test` by default:

```
!(test (+ 1 2) 3)
```

stays:

```
!(test (+ 1 2) 3)
```

Quiet assertion helpers are appropriate for explicit profile tests. Examples include `assertEqualToEval`, `assertEqualToResult`, and `assertAlphaEqualToResult`. They are not a faithful replacement for a source `test` whose printed report matters.

9 Hyperpose

`hyperpose` is PeTTa’s parallel nondeterministic surface. The default portable translation lowers it to `superpose`, preserving the nondeterministic values but not claiming to preserve parallel scheduling.

```
(hyperpose xs) -> (superpose xs')
```

When the target runtime intentionally supports `hyperpose`, users may ask the translator to preserve it:

```
./translate.sh petta2he --preserve-hyperpose input.metta output.metta
```

Such a file is no longer the default pure-portable lowering. It is a runtime-profile artifact.

10 Type Queries

The translator depends on unknown type behavior being stable. The intended behavior is:

```
!(get-type $x) ; %Undefined%
!(get-metatype $x) ; Variable
```

Returning a fresh type variable for `(get-type $x)` is unsound for translation: that variable can escape into stored type facts and later unify with concrete types, producing branch-order-dependent routes. Unknown types should remain unknown; metatypes distinguish variables.

11 Imports, Spaces, And State

Translation over imports is supported in recursive and bundle modes. Space and state operations are part of the semantic boundary because they can expose mutation order, duplicate atoms, backend indexing, and host resources.

PeTTa named state and HE native state share spellings such as `new-state`, `get-state`, and `change-state!`, but they are not the same portable contract. A PeTTa form such as:

```
!(bind! counter (new-state 0))
```

uses `counter` as a mutable cell name. HE native state allocates an explicit state handle. The PeTTa-to-HE direction therefore lowers named state through translator helpers instead of treating it as identity syntax.

The rule is simple: preserve or lower what has a specified target meaning, and classify the rest. Python handles, Git imports, MORK/PathMap handles, runtime statistics, and other host resources are not pure HE merely because the syntax is readable by an HE parser.

12 Evidence And Gates

There are several distinct evidence layers:

Gate	Evidence
<code>translate.sh -test</code>	Unit and real-file translator behavior.
<code>run_he_profile_suite.sh</code>	PeTTa <code>-he</code> runtime profile behavior.
<code>run_translator_survey.sh</code>	Corpus correctness and performance budget.
<code>check_generated_he_portability.sh</code>	Cross-engine classification.
Lean translator files	Proof-oriented common-fragment correspondence.

These gates answer different questions. A green PeTTa `-he` survey is not automatically a cross-engine proof. A helper-surface classification is not a runtime failure. A proof over the common fragment does not license arbitrary host extensions.

13 Production Criteria

A translation rule is production-ready only when:

1. its source and target semantic lanes are named;
2. positive and negative examples are recorded;
3. fresh variables are capture-avoiding;
4. generated helper names avoid source-symbol collisions;
5. observable output is preserved or explicitly classified;
6. helper surfaces are named as helpers, not HE core;
7. translator tests pass;
8. representative generated artifacts run under their intended target;
9. cross-engine differences are classified when portability is claimed.

Performance work must preserve these correctness obligations. A faster translation that changes source behavior is a regression.

14 Conclusion

The $\text{HE} \leftrightarrow \text{PeTTa}$ translator should be understood as a semantic bridge between related MeTTa profiles, not as a string rewriter between identical languages. Its most important discipline is classification: shared core behavior, portable lowering, helper surfaces, compatibility profiles, and extensions must stay distinct. With that discipline, the translator can serve both engineering use and formalization work without making false portability claims.